

1. Dynamic Memory Allocation — malloc() & calloc()

Introduction

In C language, memory can be allocated during run-time using Dynamic Memory Allocation (DMA).

Dynamic memory comes from the Heap area.

malloc() — Memory Allocation

- malloc() means Memory Allocation
- It allocates a continuous block of memory
- Memory remains uninitialized (garbage values)
- Returns void pointer

Syntax:

```
ptr = (int*) malloc(n * sizeof(int));
```

Example:

```
int *p;
```

```
p = (int*) malloc(5 * sizeof(int));
```

Memory View:

```
| ? | ? | ? | ? | ? | (garbage values)
```

calloc() — Contiguous Allocation

- Allocates multiple memory blocks
- Each block initialized to 0
- Used for arrays

Syntax:

```
ptr = (int*) calloc(n, sizeof(int));
```

Memory View:

```
| 0 | 0 | 0 | 0 | 0 |
```

Difference

<u>Feature</u>	<u>malloc()</u>	<u>calloc()</u>
<u>Full form</u>	<u>Memory Allocation</u>	<u>Contiguous Allocation</u>
<u>Blocks</u>	<u>Single</u>	<u>Multiple</u>
<u>Initialization</u>	<u>Garbage</u>	<u>Zero</u>
<u>Syntax</u>	<u>malloc(n * size)</u>	<u>calloc(n, size)</u>

2. Stack — Implementation using Array

Definition

Stack is a Linear Data Structure that follows LIFO (Last In First Out).

Basic Operations

- **push() — insert**
 - **pop() — remove**
 - **peek() — view top**
 - **isEmpty()**
 - **isFull()**
-

Algorithm: PUSH

```
if top == max-1  
    print "Overflow"  
  
else  
  
    top = top + 1  
  
    stack[top] = value
```

Algorithm: POP

```
if top == -1  
  
    print "Underflow"  
  
else  
  
    top = top - 1
```

Stack Diagram

Top -> | 30 |
| 20 |
10

☒ 3. Expression Notations — Infix, Prefix, Postfix

Definitions

<u>Type</u>	<u>Forma</u> <u>t</u>	<u>Example</u>
<u>Infix</u>	<u>A + B</u>	<u>Operators between</u> <u>operands</u>
<u>Prefi</u> <u>x</u>	<u>+ A B</u>	<u>Operators before</u>
<u>Postfi</u> <u>x</u>	<u>A B +</u>	<u>Operators after</u>

Example: Convert

Infix: A + B * C

Postfix: A B C * +

Prefix: + A * B C

4. Pointers in C

Definition

A pointer is a variable that stores address of another variable.

```
int a = 10;
```

```
int *p = &a;
```

Diagram

a = 10 at address 1000

p -> 1000

Pointers are used in:

- Dynamic memory allocation
- Arrays & strings
- Linked List, Trees, Graphs
- Function data passing

5. Recursion

Definition

Recursion occurs when a function calls itself.

Factorial Example

```
int fact(int n){  
    if(n==0) return 1;  
    return n * fact(n-1);  
}
```

Recursion Tree (n=3)

```
fact(3)  
→ 3 * fact(2)  
→ 2 * fact(1)  
→ 1 * fact(0)
```

6. Tower of Hanoi

Rules

- Move one disk at a time
- Never place bigger disk on smaller

Algorithm

```
T0H(n, A, B, C):  
    if n==1: move A → C  
    T0H(n-1, A, C, B)  
    move A → C
```

TOH(n-1, B, A, C)

Diagram (3 disks)

Start

A B C

| 1 |

| 2 |

| 3 |

Goal

A B C

 | 1 |

 | 2 |

 | 3 |

✓ 7. Singly Linked List

Node Structure

data | next

Linked List Representation

[10 | *] -> [20 | *] -> [30 | NULL]

Insertion Algorithms

At Beginning

new->next = head

head = new

At End

temp = head

while temp->next != NULL

temp = temp->next

temp->next = new

Deletion

Delete First Node

temp = head

head = head->next

free(temp)

8. Polynomial Representation Using Linked List

Example Polynomial

$3x^3 + 2x^2 + 5x + 1$

Node Format

[coef, pow] -> [coef, pow]

Diagram

[3,3] -> [2,2] -> [5,1] -> [1,0] -> NULL

Advantages

- Efficient storage
- Easy insertion
- Used in symbolic computation



Final Summary Table

<u>Topic</u>	<u>Key Points</u>
<u>malloc vs calloc</u>	<u>Dynamic Memory</u>
<u>Stack</u>	<u>LIFO, push/pop</u>
<u>Infix-Prefix-Post fix</u>	<u>Expression conversion</u>
<u>Pointers</u>	<u>Store address</u>

<u>Recursion</u>	<u>Function calls itself</u>
<u>Tower of Hanoi</u>	<u>Recursive puzzle</u>
<u>Linked List</u>	<u>Node (data, next)</u>
<u>Polynomial LL</u>	<u>(coef, power) nodes</u>

Malloc and calloc(memory allocation):-

function:- malloc and calloc is the function library use for the dynamic memory allocation allow the program memory obtain during the program

Malloc:-malloc is the allocated the is single block of memory allocation

Example of : - `int *p = (int *)malloc(5 * sizeof(int));`

calloc : - malloc is used to multiple memory allocations

Example of : - `int *p = (int *)calloc(5, sizeof(int));`

Stack:- Stack is linear data structure follow the (lifo) last in out and using the either the array index base

1. STACK (Array Implementation)

Push Operation

Algorithm:

START

IF top == maxSize - 1

 PRINT "Stack Overflow"

ELSE

 top = top + 1

 stack[top] = element

ENDIF

STOP

Pop Operation

START

IF top == -1

 PRINT "Stack Underflow"

ELSE

element = stack[top]

 top = top - 1

 PRINT element

ENDIF

STOP

Peek

IF top == -1

 PRINT "Stack Empty"

ELSE

 PRINT stack[top]

ENDIF

Diagram

| 40 | <- TOP

| 30 |

| 20 |

| 10 |



2. STACK (Linked List Implementation)

Push

START

Create newNode

newNode.data = value

newNode.next = top

top = newNode

STOP

Pop

START

IF top == NULL

PRINT "Underflow"

ELSE

temp = top

top = top->next

free(temp)

ENDIF

STOP



3. SINGLY LINKED LIST

Insert at Beginning

Create newNode

newNode.data = value

newNode.next = head

head = newNode

Insert at End

Create newNode

newNode.data = value

newNode.next = NULL

IF head == NULL

 head = newNode

ELSE

 temp = head

 WHILE temp.next != NULL

 temp = temp.next

 ENDWHILE

 temp.next = newNode

ENDIF

Insert in Middle (After Given Node)

Create newNode

newNode.data = value

newNode.next = prevNode.next

prevNode.next = newNode

Delete from Beginning

```
IF head == NULL  
    PRINT "List Empty"  
  
ELSE  
  
    temp = head  
  
    head = head->next  
  
    free(temp)  
  
ENDIF
```

Delete from End

```
IF head == NULL  
  
    PRINT "List Empty"  
  
ELSE IF head.next == NULL  
  
    free(head)  
  
    head = NULL  
  
ELSE  
  
    temp = head  
  
    WHILE temp.next.next != NULL  
  
        temp = temp.next  
  
    ENDWHILE  
  
    free(temp.next)
```

temp.next = NULL

ENDIF

Delete Specific Node (value based)

IF head.data == value

head = head->next

ELSE

temp = head

WHILE temp.next != NULL AND temp.next.data != value

temp = temp.next

ENDWHILE

node = temp.next

temp.next = node.next

free(node)

ENDIF

Linked List Diagram

HEAD

↓

[10 | *] -> [20 | *] -> [30 | NULL]

✓ 4. QUEUE (Array)

Enqueue

IF rear == max-1

 PRINT "Overflow"

ELSE

 rear++

 queue[rear] = value

ENDIF

Dequeue

IF front > rear

 PRINT "Underflow"

ELSE

 value = queue[front]

 front++

ENDIF

Queue Diagram

Front → | 10 | 20 | 30 | ← Rear

✓ 5. POLYNOMIAL USING LINKED LIST

Node Structure

coef, power, next

Insert in Descending Power

Create newNode

IF list empty OR power of newNode > head.power

 newNode.next = head

 head = newNode

ELSE

 temp = head

 WHILE temp.next != NULL AND temp.next.power > newNode.power

 temp = temp.next

 ENDWHILE

 newNode.next = temp.next

 temp.next = newNode

ENDIF

Polynomial Diagram

[3,3] -> [2,2] -> [5,1] -> [1,0] -> NULL

✓ 6. TOWER OF HANOI Algorithm

T0H(n, source, helper, destination)

IF n == 1

```

    PRINT Move disk from source to destination

ELSE

    TOH(n-1, source, destination, helper)

    PRINT Move disk from source to destination

    TOH(n-1, helper, source, destination)

ENDIF

```

Diagram (3 disks)

A B C

| 1 |

| 2 |

| 3 |

Goal:

A B C

_____ | 1 |

_____ | 2 |

_____ | 3 |

✓ 7. SIMPLE RECURSION EXAMPLE (Factorial)

fact(n)

IF n == 0

```
    RETURN 1  
  
ELSE  
  
    RETURN n * fact(n-1)  
  
ENDIF
```

Final Topic-wise Diagram Summary

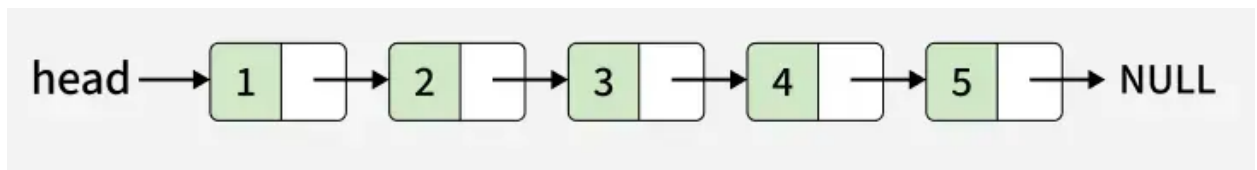
<u>DSA Type</u>	<u>Structure</u>
<u>Stack</u>	<u>LIFO</u>
<u>Queue</u>	<u>FIFO</u>
<u>Linked List</u>	<u>Node → pointer to next node</u>
<u>Polynomial</u>	<u>(coef, power) linked nodes</u>
<u>Tower of Hanoi</u>	<u>Recursive disk movement</u>
<u>Recursion</u>	<u>Function calls itself</u>

link list data structure:-

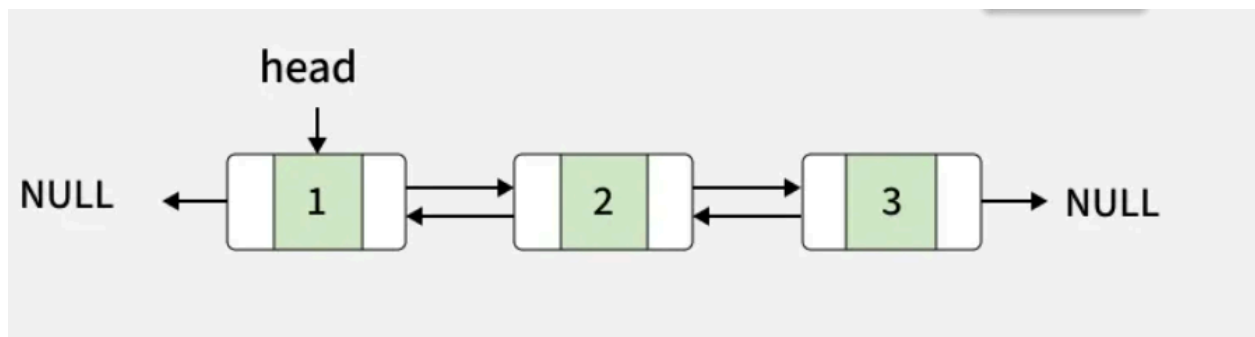
Link list is the a data structure in which elements called nodes are stored in a non- contiguous memory location e, h node having a data pointer the next node

Types of the link list 🍌

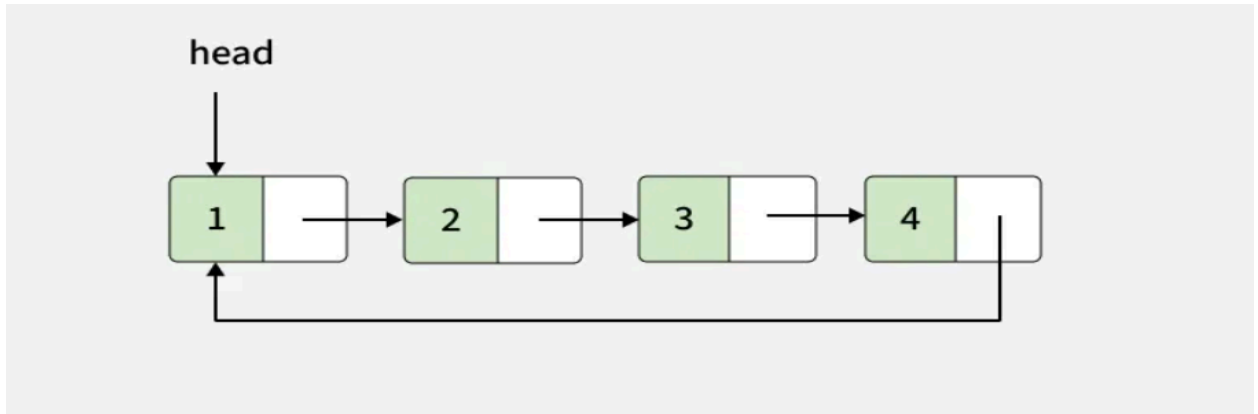
1. Single linked lists (sll): each node pointer to the next node



2. Doubly link list (dll) :- Each node store the dta previous node and next node called of the doubly link list



3. Circular link list (c ll): in circular link lis t last node point to the first node and that head of the node point ot last node



Q difference between array and link list

Array vs Linked List		
Aspect	Array	Linked List
Memory Layout	Stored in a contiguous block of memory	Stored in non-contiguous memory, nodes linked with pointers
Access Time	$O(1)$ random access using index	$O(n)$ sequential access (must traverse nodes)
Insertion/Deletion	$O(n)$ (shifting required), except at end where it can be $O(1)$ if space available	$O(1)$ if node reference is known, otherwise $O(n)$ to locate the position
Search	$O(n)$ linear search, $O(\log n)$ with binary search (if sorted)	$O(n)$ sequential search only
Memory Overhead	No extra memory overhead	Extra memory needed for pointers in each node
Cache Friendliness	High, due to contiguous storage	Low, as nodes may be scattered in memory
Size Flexibility	Fixed size (unless dynamically resized, e.g., vector in C++)	Dynamic size, grows or shrinks as needed

Introduction to Data Structures & Basic Terminologies

1.1 What is Data?

- **Data:** Raw facts and figures without any context.
 - o E.g., 10, John, 2025
- **Information:** Processed data that has meaning.
 - o E.g., "John scored 10 marks in 2025."

The topic is the time complexity

Exmple : constant complexity:

Q. Write the program square of the number and size of the data type of variable

```
#include <stdio.h>

int square(int n) {
    printf("    %d bytes\n", sizeof(int));
    printf("    %d bytes\n", sizeof(n));
    return n * n;
}

int main() {
    int num;
    printf("enter a number: ");
    scanf("%d", &num);
    int value = square(num);
    printf("the square of %d is %d\n", num, value);
    return 0;
}
```

Output size of the array of data types 👍

enter a number: 50

4 byt

4 byteThe square of 50 is 2500

#Linear complexity

#Sorting:

Buble sorting 👍

The existing document mentions "Buble sorting 👍" under the section for "Linear complexity" and "Sorting," indicating a planned inclusion of the Bubble Sort concept.

```
#include <stdio.h>

int main () {

    int i , n , j , temp;

    printf("  Enter   size of  array");

    scanf ( "%d", &n);

    int a[n];

    printf("  Enter  value in  array ");

    for ( int  i  = 0; i< a; i++){

        scanf  ( "%d",  &a[i]);

    }

    for(  i = n; i >= 0; i-- ){

        for( j=0; j<i-1; j++ ){

            if (a [j] > a [j+1]) {

                temp = a[j];

                a[j] = a[j+1];

                a[j+1] = temp;

            }

        }

    }

}
```



```

    }

}

printf ( " Sorted array is : \n" );

for ( i = 0; i < n; i++ ) {

    printf ( "%d  ", a[i] );

}

}

```

Explanation of Bubble Sort

Bubble Sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted. It gets its name because smaller or larger elements "bubble up" to their correct position with each pass.

Complexity of Bubble Sort

Case	Time Complexity
Worst Case	$O(n^2)$
Average Case	$O(n^2)$

Case	Time Complexity
Best Case	$O(1)$ (if array is already sorted)

The space complexity for Bubble Sort is **$O(1)$** as it only uses a few extra variables for comparison and swapping.

- **Definition:** A data structure is a method of organizing and storing data in a computer so that it can be used efficiently.
- **Need:**
 - o Easy access and modification
 - o Efficient memory usage
 - o Code clarity and modularity

1.3 Basic Terminologies

Term	Definition	Example
Data Item	A single unit of data	5, 'A'
Group Item	Data composed of sub-items	Name = (First + Last)
Record	Collection of related data items	Student = (Name, Roll, Marks)
File	Collection of records	Student file
Field	A single attribute in a record	Roll number
Entity	Object with real-world existence	Student, Book
Attribute	Property of an entity	Name, ID, Age
Key	Uniquely identifies a record	Student Roll No

1.4 Types of Data

- **Primitive Data Types:** Integer, Float, Char, Boolean
- **Non-Primitive:** Arrays, Lists, Stacks, Trees, Graphs

1.5 Classification of Data Structures

1. Linear Data Structures

- o Elements arranged in sequence
- o Examples: Arrays, Linked Lists, Stacks, Queues

2. Non-linear Data Structures

- o Elements arranged hierarchically or graph-based
- o Examples: Trees, Graphs

1.6 Abstract Data Types (ADT)

- **ADT:** A model defined by its behavior (operations), not by implementation
- Example: Stack ADT – operations: push(), pop(), peek()

2.1 Elementary Data Organization

- **Memory Concepts**

- o Memory = a linear collection of cells with addresses
- o Data stored in consecutive cells or via pointers

2.2 Data Structure Operations

Operation	Description
Traversal	Visit every element
Insertion	Add new element
Deletion	Remove element
Searching	Find specific element
Sorting	Arrange in order

Operation	Description
Merging	Combine two data sets

2.3 Types of Data Structures

A. Linear Data Structures

Structure	Description	Access Type
Array	Fixed-size, index-based	Random Access
Linked List	Nodes linked using pointers	Sequential
Stack	LIFO (Last In First Out)	Access only top
Queue	FIFO (First In First Out)	Front for deletion, rear for insertion

B. Non-Linear Data Structures

Structure	Description
Trees	Hierarchical data, used in filesystems, databases
Graphs	Collection of nodes and edges, used in networks

2.4 Static vs Dynamic Structures

Type	Static	Dynamic
Size	Fixed at compile time	Can grow/shrink at runtime

Type	Static	Dynamic
Example	Array	Linked List
Memory	Contiguous	Non-contiguous
Flexibility	Low	High

Algorithms and Complexity

3.1 What is an Algorithm?

- A step-by-step procedure to solve a problem
- Must be:
 - **Finite:** Has an end
 - **Definite:** Clearly defined steps
 - **Input:** Zero or more inputs
 - **Output:** At least one output
 - **Effective:** Feasible steps

3.2 Characteristics of a Good Algorithm

- Correctness
- Efficiency (time and space)
- Readability
- Robustness
- Scalability

3.3 Algorithm Design Techniques

- **Divide and Conquer** – e.g., Merge Sort
- **Greedy Method** – e.g., Dijkstra's Algorithm
- **Dynamic Programming** – e.g., Fibonacci, Knapsack
- **Backtracking** – e.g., N-Queens Problem
- **Brute Force** – e.g., Linear Search

What is Array?

Array is a linear data structure where all elements are arranged sequentially. It is a collection of elements of **same data type** stored at **contiguous memory locations**.

Basic terminologies of Array

- **Array Index:** In an array, elements are identified by their indexes. Array index starts from 0.
- **Array element:** Elements are items stored in an array and can be accessed by their index.
- **Array Length:** The length of an array is determined by the number of elements it can contain.

Memory representation of Array

In an array, all the elements are stored in contiguous memory locations. So, if we initialize an array, the elements will be allocated sequentially in memory. This allows for efficient access and manipulation of elements.

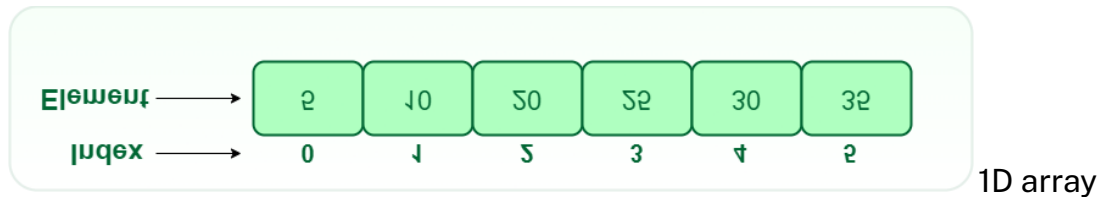
Types of Arrays

Types of Arrays on the basis of Dimensions

There are majorly three types of arrays on the basis of dimensions:

1. One-dimensional array (1-D arrays):

You can imagine a 1d array as a row, where elements are stored one after another.



Syntax for Declaration of Single Dimensional Array

Below is the syntax to declare the single-dimensional array

```
data_type array_name[array_size];
```

where,

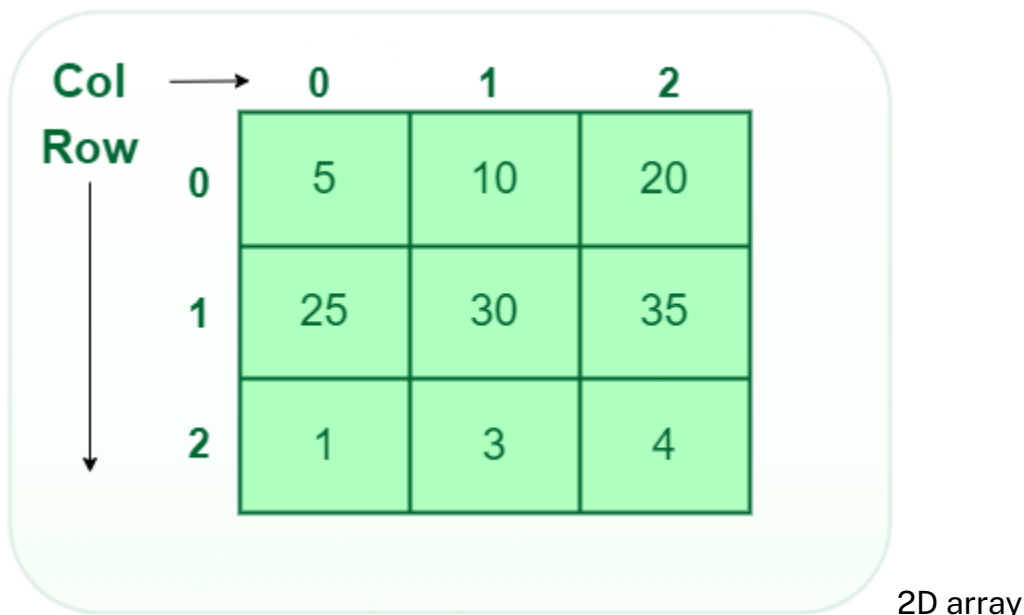
- **data_type:** is a type of data of each array block.
- **array_name:** is the name of the array using which we can refer to it.
- **array_size:** is the number of blocks of memory array going to have.

For Example

```
int nums[5];
```

2. Two-dimensional (2D) array:

Multidimensional arrays can be considered as an array of arrays or as a matrix consisting of rows and columns.



Syntax for Declaration of Two-Dimensional Array

Below is the syntax to declare the Two-dimensional array

```
data_type array_name[sizeof_1st_dimension][sizeof_2nd_dimension];
```

where,

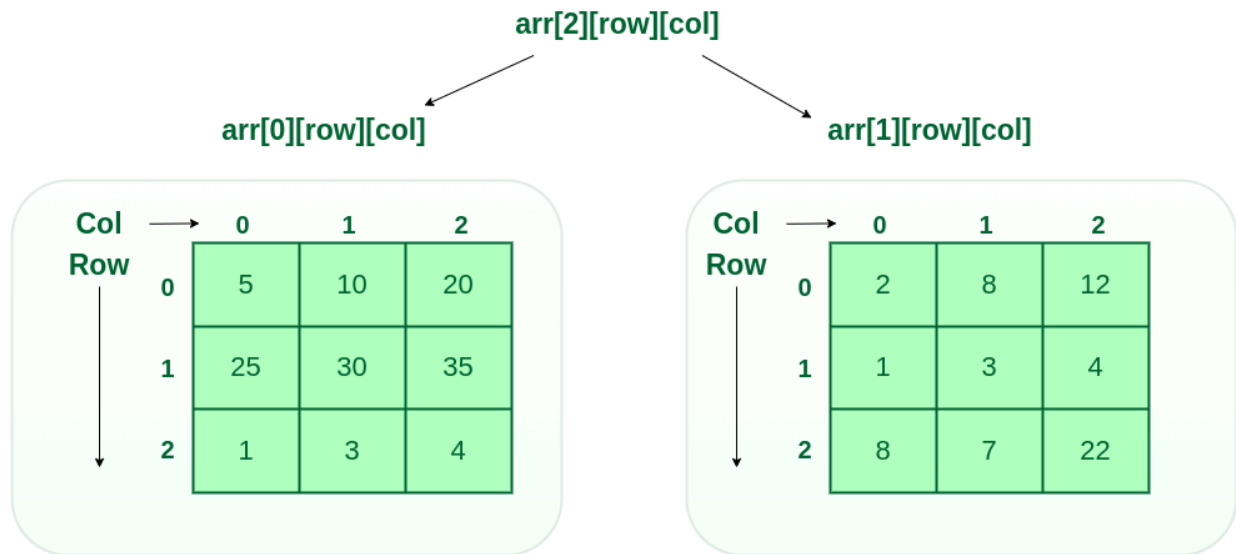
- **data_type:** is a type of data of each array block.
- **array_name:** is the name of the array using which we can refer to it.
- **sizeof_dimension:** is the number of blocks of memory array going to have in the corresponding dimension.

For Example

```
int nums[5][10];
```

3. Three-dimensional array:

A 3-D Multidimensional array contains three dimensions, so it can be considered an array of two-dimensional arrays.



3D array

Syntax for Declaration of Three-Dimensional Array

Below is the syntax to declare the Three-dimensional array

`data_type`

```
array_name[sizeof_1st_dimension][sizeof_2nd_dimension][sizeof_3rd_dimension];
```

where,

- **data_type:** is a type of data of each array block.
- **array_name:** is the name of the array using which we can refer to it.
- **sizeof_dimension:** is the number of blocks of memory array going to have in the corresponding dimension.

Operations on Array

1. Array Traversal
2. Insertion in Array
3. Deleting the Array
4. Searching in Array

Searching Algorithms

Searching algorithms are essential tools in computer science used to locate specific items within a collection of data. In this tutorial, we are mainly going to focus upon searching in an array. When we search an item in an array, there are two most common algorithms used based on the type of input array.

- **Linear Search** : It is used for an unsorted array. It mainly does one by one comparison of the item to be search with array elements. It takes linear or $O(n)$ Time.
- **Binary Search** : It is used for a sorted array. It mainly compares the array's middle element first and if the middle element is same as input, then it returns. Otherwise it searches in either left half or right half based on comparison result (Whether the mid element is smaller or greater). This algorithm is faster than linear search and takes $O(\log n)$ time.

Sparse Matrix and its representations

A **matrix** is a two-dimensional data object made of m rows and n columns, therefore having total $m \times n$ values. If most of the elements of the matrix have **0 value**, then it is called a sparse matrix.

Why to use Sparse Matrix instead of simple matrix ?

- **Storage:** There are lesser non-zero elements than zeros and thus lesser memory can be used to store only those elements.
- **Computing time:** Computing time can be saved by logically designing a data structure traversing only non-zero elements..

Example:

```
0 0 3 0 4
0 0 5 7 0
0 0 0 0 0
0 2 6 0 0
```

Representing a sparse matrix by a 2D array leads to wastage of lots of memory as zeroes in the matrix are of no use in most of the cases. So, instead of storing zeroes with non-zero elements, we only store non-zero elements. This means storing non-zero elements with **triples- (Row, Column, value)**.

Sparse Matrix Representations can be done in many ways following are two common representations:

1. Array representation
2. Linked list representation

Difference between Linear Search and Binary Search

Feature	Linear Search	Binary Search
Definition	Checks elements one by one until the target is found.	Repeatedly divides the sorted list in half to find the target.
Works On	Both sorted and unsorted data.	Only sorted data.
Approach	Sequential comparison.	Divide-and-conquer (check middle, then half list).
Time Complexity	Best: $O(1)$ Worst: $O(n)$	Best: $O(1)$ Worst: $O(\log n)$
Space Complexity	$O(1)$	Iterative: $O(1)$ Recursive: $O(\log n)$
Efficiency	Slow for large datasets.	Very fast for large datasets (if sorted).
Example (List = [2, 5, 7, 10, 15, 20], Target = 15)	Steps: 2 → 5 → 7 → 10 → 15 → Found in 5 steps	Steps: Check 7 → (15 > 7) → New list [10, 15, 20] → Check 15 → Found in 2 steps

Linear Search is a simple searching technique where each element in the list is checked one by one until the target element is found or the end of the list is reached. It works on both **sorted and unsorted data**, but it is not efficient for large datasets. For example, if we have a list [2, 5, 7, 10, 15, 20] and we want to find 15, the search will check 2 → 5 → 7 → 10 → 15, and the element will be found in the **5th step**. The time complexity is **$O(n)$** in the worst case, since in the worst situation all elements must be checked.

Binary Search, on the other hand, is a much faster technique but it requires the list to be **sorted**. It follows a divide-and-conquer approach by repeatedly checking the middle element and then deciding whether to search in the left half or the right half of the list. For example, in the same list [2, 5, 7, 10, 15, 20], if we search for 15, the algorithm first checks the middle element 7. Since 15 is greater than 7, it searches the right half [10, 15, 20]. Then it checks the middle of this half, which is 15, and finds the element in just **2 steps**. The time complexity is **$O(\log n)$** in the worst case, making it very efficient for large, sorted datasets.

Complexity

What is Time Complexity?

- **Definition:** Time complexity measures **how much time (steps or operations)** an algorithm takes to run as the input size grows.
 - It does not measure actual clock time, but the **number of operations** in terms of input size **n**.
 - Example: In **Linear Search**, if we have 100 elements, in the worst case we may check all 100 → **O(n)**.
-

◆ What is Space Complexity?

- **Definition:** Space complexity measures **how much extra memory (RAM)** an algorithm needs apart from the input data.
 - This includes variables, data structures, recursion stack, etc.
 - Example: **Binary Search (recursive)** requires extra memory for function calls, while **Linear Search** just needs a variable → **O(1)**.
-

◆ Difference between Time Complexity and Space Complexity

Aspect	Time Complexity	Space Complexity
Meaning	Amount of time/steps an algorithm takes to run based on input size.	Amount of memory/space an algorithm uses during execution.
Measured In	Number of operations (comparisons, loops, recursive calls).	Amount of extra memory (variables, data structures, call stack).
Goal	Make algorithm run faster .	Make algorithm use less memory .
Example	Linear Search takes O(n) time in worst case.	Linear Search takes O(1) space.

Aspect	Time Complexity	Space Complexity
Trade-off	Faster algorithms may sometimes need more memory.	Memory-efficient algorithms may sometimes run slower.

Static vs Dynamic Data Allocation

Static Memory Allocation

- **Definition:** Memory for variables is allocated **at compile time** (before the program runs).
- **Size** of memory must be known in advance.
- Once allocated, the memory **cannot be changed** during execution.
- Allocated memory is usually stored in the **stack**.

Example in C:

```
int arr[10]; // array of fixed size 10
```

Here, memory for 10 integers is reserved at compile time. You cannot increase or decrease the size later.

Dynamic Memory Allocation

- **Definition:** Memory is allocated **at runtime** (when the program runs).
- The size of memory can be decided by the program/user **during execution**.
- Allocated memory can be **resized** or **freed** when no longer needed.
- Memory is allocated from the **heap**.

Example in C:

```
int *arr;
arr = (int*) malloc(10 * sizeof(int)); // array of size 10 at runtime
```

Here, memory for 10 integers is created at runtime. You can later reallocate it using `realloc()` or free it using `free()`.

◆ Difference between Static and Dynamic Memory Allocation

Aspect	Static Allocation	Dynamic Allocation
When Allocated	At compile time	At runtime
Flexibility	Fixed size, cannot be changed	Flexible size, can grow/shrink
Memory Location	Stack	Heap
Speed	Faster (pre-decided at compile time)	Slower (decided at runtime)
Memory Usage	May waste memory if array is larger than needed	Efficient use, allocate only what is required
Example	<code>int arr[50];</code>	<code>int *arr = (int*)malloc(50*sizeof(int));</code>

✅ In short:

- **Static allocation** = Fixed size, fast, but less flexible.
- **Dynamic allocation** = Flexible size, efficient, but slightly slower.

